FLASK

It is a lightweight, open-source Python web framework used to develop web Applications & REST API's.

It provides essential tools for handling requests, routing URL's and returing response.

→ Uses of Flask

- **Web Development**: Build websites and web Applications
- **API Development**: Create REST API's for communication betn Applications.
- **ML Model Deployment**: Deploy trained machine learning models through API's.
- **Backend Development**: Handle requests, business logic, & Response
- **Form Handling**: Receive and process user-submitted data.
- **Database Integration**: connect Applications with databases like PostgreSQL & MySQL.
- **Prototyping**: Quickly build & test web Applications.
- **Microservices**: Develop lightweight, independent backend services.

→ Installing Flask

- **step: 1** - create a project folder & enviroment.

```
mkdir flask_demo
cd flask_demo
conda create -p./venv python=3.9 -y
conda activate ./venv
```

- **Step: 2** - Install flask

create requirement.txt:

```
flask
```

Install Dependencies:

```
pip install -r requirement.txt
```

- **step: 3** - creating our first Flask App

create app.py:

```
from flask import Flask
app = Flask (__name__)
if __name__ == '__main__':
    app.run (debug =True)
```

* Flask :- Imports the Flask framework
* Flask(__name__) :- Creates the application instance
* app.run( ) :- Starts the Development server.
* debug =True :- Enables debugging and automatic reloading.
* We got URL / Default URL :- http://127.0.0.1:5000/

- **Step : 4** :- Run the Application

```
python app.py
```

→ Flask Routing

"Routing" is a process of connecting a URL to a specific Python function.

When a client visits a URL, flask executes the associated function & returns its response.

↳ Example

```
@app.route('/')
def welcome():
    return "<h1> Welcome from Aniket</h1>"
@app.route('/index')
def index():
    return "<h2>Index Page</h2>"
```

HTML response

- @app.route - connects a URL with a function.
- / - The root or home endpoint/home
- /index - A custom endpoint path /Index Page.
- View function - A python function that processes a request & returns a response
- HTML Response - Flask can return HTML content that the browser renders as a webpage.

→ HTTP GET and POST

GET	POST
(1) Retrieves Data	(1) submits Data
(2) Parameters often appear in the URL	(2) Data is commonly sent in the request body
(3) Used to fetch pages or records	(3) Used to submit forms or create resources

└→ Example:

```
from flask import request

@app.route('/form', methods =['GET', 'POST'])
def form_handler():
    if request.method == 'POST':
        return "Form submitted"
    return "Form page"
```

- request.method - Identifies the HTTP method.
- methods - specifies which methods the route accepts.
- request - Flask's request object, which provides access to Incoming request Information.

→ Flask Variable Rules

"Variable Rules" allows a URL to Accept dynamic values.

These values are passed to the corresponding Python view Python view function as Argument.

↳ Example-

```
@app.route('/success/<int:score>')
def success(score):
    return f"Your score is {score}"
```

- Example URL - /success/85
- Output - Your score is 85

↳ Common URL converters:

- string : Accepts texts without slashes; this is default
- int : Integer
- float : Decimal no.
- path : Text that can include slashes

* Use `str()` or f-strings when displaying no. inside strings.

→ HTML Templates and Form Data

(A) Rendering HTML templates

"Templates" are HTML files used to structure webpage seperately from Python Backend logic.

- Flask uses the Jinja2 template engine to insert dynamic values into these files.
- create files in this structure:

```
flask_demo/
├── app.py
└── templates/
    └── form.html
```

↳ **Example:**

```
from flask import render_template

@app.route('/form')
def form():
    return render_template('form.html')
```

- `render_template()` : Loads and renders an HTML template.
- `templates/` : Flask's default directory for HTML templates.
- **Jinja2**: A Template engine that supports dynamic variables, loops & conditional statements inside HTML.

*Example Jinja2 Expression -

```
<h1> Your score is {{score}} </h1>
```

{{score}} - displays the value supplied by the backend.

(B) Extracting form Data

"Form Handling" allows a flask application to receive & process information entered by users through HTML forms.

→ **Example -**

```
From flask import request
@app.route('/submit', methods=['POST'])
def submit():
    maths = float(request.form['maths'])
    science = float(request.form['science'])
    return f"Maths: {maths}, Science: {science}"
```

- **request.form** - accesses submitted form fields.
- **float()** - converts input strings into decimal numbers.
- **{{score}}** - displays a Backend variable in a Jinja2 templates.
- **request.form['maths']** - Retrieves the value associated with the "maths" field.

→ **URL Building & Redirection**

- **url_for()** - generates a URL using a view function's name.
- **redirect()** - sends the browser to another URL.

↳ Example -

```
from flask import redirect, url_for

@app.route('/go')
def go():
    return redirect(url_for('success', score=85))
```

- This redirects the user to the "success" route with a score of 85.

↳ Use case - Redirecting users to a success or failure page after processing examination marks or submitting a form.

→ Creating a Flask REST API

"REST API" allows applications to exchange data through HTTP requests, commonly using JSON.

↳ Important concepts -

- Endpoint : A URL through which a client accesses a particular API operation.
- request.get_json() : Parses incoming JSON request data into Python objects.
- jsonify() : converts Python data into a JSON HTTP response.

- **Postman** - It's a tool for sending HTTP requests and inspecting API responses without developing a frontend.

- **JSON** - It's a lightweight data interchange format that represents objects, array, strings, numbers & other values.

↳ **Example**

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api', methods=['POST'])
def calculate_sum():
    data = request.get_json()
    total = float(data['a']) + float(data['b'])
    return jsonify({'result': total})

if __name__ == '__main__':
    app.run(debug=True)
```

- **Example request body**:

  Json

```
{
  "a": 10
  "b": 20
}
```

- **Response**

```
{
  "result": 30.0
}
```